From Domain to Action:
A Cohesive Planning Pipeline for Game AI in Unity Engine

Marek Marchlewicz

Independent Researcher
marek@aiingames.com

Abstract

Automated planning has been available as a technique for game AI for over two decades, yet adoption outside well-resourced studios remains limited. The barrier is not algorithmic. Forward-search planning over STRIPS-style domains is sufficiently performant for real-time game applications. Rather, the challenge is infrastructural: no complete, engine-integrated toolchain addresses the four requirements developers must solve to deploy planning in a game. Those requirements are authoring a domain model, formulating planning problems from live game state, running the planner within the frame time budget, and dispatching the resulting plan to game-engine actions. This paper presents a cohesive system for the Unity engine that addresses all four, in which each component's output is the next component's input: domains are authored in a visual editor, problems are extracted from live game objects, planning requests run through a priority queue pumped once per frame from the player loop, and plans are dispatched to developer-supplied action handlers. Planning is provided by two forward planners that share one A* engine: a general dictionary planner, and a compiled bit-vector planner. The bit-vector version plans in about 0.1 ms, comparable to the ReGoap library on the smallest domain, and orders of magnitude faster as domains grow. The paper describes the design of each component, and the failure modes encountered during development and evaluation.

Code — https://github.com/AI-In-Games/Unity-Planner

1 Introduction

In 2006, Orkin described a planning system deployed in a commercial first-person shooter (Orkin 2006). The Goal-oriented Action Planning aka GOAP, used a symbolic world-state representation and A* search over actions to generate behavior plans at runtime. Although relatively straightforward by planning standards, it produced NPC behavior that felt qualitatively different from traditional scripted AI. The paper was widely read, and similar techniques were adopted by several studios, however, they never became mainstream.

The reason it stalled is not that GOAP stopped working. It is that every team that wanted to use it had to build the same infrastructure from scratch: a way to define world state predicates, a way to ground actions at runtime, a way to run the planner on a background thread without starving the render loop, and a way to connect planner output to movement, animation, and audio systems. Each of these is a solved problem in isolation, but solving all four together, inside the conventions of a specific game engine, takes enough engineering time that many teams abandon the approach or reduce it to a hand-coded finite-state machine. The algorithm was never the bottleneck. The toolchain was.

A presented solution is designed to eliminate that toolchain barrier. It has four components corresponding to the four infrastructure problems above, and the contribution is their cohesion: each component is designed so that its output format is the input format of the next, with no glue code required from the developer for the common case.

This paper makes the following contributions:

1. A description of the four integration problems that must be solved to ship planning in a game (authoring a domain, formulating problems from live game state, running the planner without disrupting the frame budget, and dispatching plans to engine actions), together with the design decisions I made for each.
2. A cohesive implementation, grounded in real Unity APIs and best practices, in which the four stages run as one wired pipeline: requests enter a priority queue, the engine pumps one per frame from the player loop, the planner runs, and the result is dispatched to developer-supplied action handlers, with minimal scripting required for the common case.
3. A planner architecture in which two forward planners, a dictionary planner and a compiled bit-vector planner, share a single A* core, but differ in their state representation, each usable with one of two heuristics. The dictionary planner doubles as a controlled baseline that isolates what bit-vector compilation contributes.
4. Empirical results on three game domains and on a scalable benchmark, separating grounding, compilation, and search, and comparing against the ReGoap library.

It also provides an honest account of the failure modes and limitations encountered, which might be useful to the community as the successful design decisions.

2 Background

2.1 Planning in Games

Orkin's GOAP formulation (2006) remains one of the most influential planning architectures in commercial game AI. Its compact symbolic state representation and lightweight planning search made it practical on mid-2000s hardware. Subsequent commercial work increasingly adopted Hierarchical Task Network (HTN) planning, which provides stronger designer control through explicit task decomposition at the cost of more complex domain authoring. HTN-based systems have been reported in several commercial titles, including the Killzone series, Transformers: Fall of Cybertron, and later Guerrilla Games productions built on the Decima engine. Surveys of commercial planning systems, such as Neufeld et al. (2017), indicate that both GOAP and HTN variants have seen successful deployment, while also highlighting recurring challenges in authoring, debugging, maintenance, and tooling support.

Community libraries for Unity (ReGoap, Fluid HTN) address portions of the infrastructure problem: ReGoap (ReGoap 2016) provides a runtime GOAP system driven by C# implementations; Fluid HTN provides an HTN framework with a C# builder. Neither provides visual domain authoring or PDDL serialization. Neither addresses the game loop integration problem systematically.

Unity Technologies showcased an experimental planning package (com.unity.ai.planner, preview 0.3) with inspector editing tools and a runtime planner, discontinued before reaching a stable release (Unity Technologies 2021). It remains the closest prior attempt at an engine-native planning toolchain.

Both major engines have since invested in AI authoring tools. Unreal Engine ships behavior trees with a visual editor and introduced StateTree in Unreal Engine 5 (2022), a hierarchical state machine with parallel state support. Unity acquired a Bolt VS and shipped it as Unity Visual Scripting in 2021. Neither addresses planning: behavior trees and state machines are reactive rather than goal-directed, and visual scripting replaces code authoring rather than providing domain representation for a planner.

2.2 Planning Tooling

The PDDL ecosystem includes several authoring-adjacent tools: VAL (Howey et al. 2004) validates domain and problem files; the Planning.Domains editor provides a web-based PDDL text editor with syntax highlighting. None of these are integrated into a game engine, and all target users are comfortable with formal planning formalisms. The planning community has long identified domain authoring as a barrier to deployment, but the game-specific variant of this problem has received less attention. De Pellegrin (2020) presents PDSim, which uses the Unity engine to visualize and animate PDDL plan execution in 3D, demonstrating Unity's suitability as a platform for planning tooling; PDSim targets the knowledge engineering use case (helping domain authors inspect plan behavior) rather than the game AI runtime integration addressed by this paper.

2.3 Real-Time Planning

Planning under time constraints has been studied in the context of anytime planners (Likhachev et al. 2004) and online planning (Koenig and Likhachev 2002). The game AI literature addresses this primarily through domain restriction (keeping domains small and well-typed) rather than through algorithmic modification. van der Sterren (2014) described packing GOAP world state into integer bitmasks, which was the inspiration for the bit-vector compilation approach.

While planning latency is a real constraint, players accept when NPCs react with a slight delay, comparable to or greater than human reaction time; instant replanning would read as inhuman rather than intelligent. A useful analogy is NavMesh pathfinding: navigation itself is fast, but computing a path on a large map is time-consuming and is therefore executed asynchronously, with the NPC continuing to move while the result arrives. Planning fits the same model.

2.4 Large Language Models and Game AI

The rapid adoption of Large Language Models (LLMs) since 2022 has prompted the question of whether symbolic planning is still necessary for game AI. Park et al. (2023) demonstrated LLM-driven agents capable of social simulation, and substantial work explores generating PDDL domain content from natural language. These results are relevant to authoring workflows but do not address the runtime constraints that motivate this paper. For context, LLM inference is often measured in seconds compared to the sub-millisecond frame budget in games. Valmeekam et al. (2023) evaluate LLMs on classical planning benchmarks and find they fail at non-trivial scale, characterizing their output as approximate retrieval rather than search; Orkin (2025) observes that wrapping an LLM with the scaffolding needed for plan-safe output (state tracking, action validation, plan repair) reconstructs the symbolic planning stack at greater cost.

Developer sentiment toward generative AI is ambivalent, even as personal usage has grown. In the GDC State of the Industry survey, the share of developers reporting a net-negative industry impact rose from 18% in 2024 to 52% in 2026, while only 7% report a net-positive view (GDC 2026). Player communities have associated AI-generated content with quality reduction, and AI-generated voice performance has become a focus of active labor disputes. The planning described in this paper represents a “hand-authored” approach: domains,

predicates, and actions are explicitly specified by the developer, and the planner executes deterministic search over those rules.

Certain LLM integration to the system could be added to this solution at the later stage should it be required to, for example, generate domain files.

3 System Overview

The system has four components, and each component's output is the next one's input, so the four stages form a single pipeline (Figure 1).

- **Domain Editor and DomainAsset -** defines predicates and action schemas; result is a *DomainAsset* that serializes to JSON and optionally exports to PDDL.
- **Grounding and compilation** - For a given problem's objects, the action schemas are grounded once. The bit-vector planner additionally compiles the grounded domain into a *CompiledDomain*, a fixed-width *ulong[]* bit-vector per state with precomputed bitmasks for each action's positive and negative preconditions and its add and delete effects. The compiled form is cached and reused across replans over the same objects.
- **PlannerService and the planner** - Agents submit a *PlanningProblemDefinition* with a numeric priority to a thread-safe priority queue. A player-loop hook pumps one request per frame, the planner runs forward A* synchronously, and the *PlanResult* is delivered to the request's callback. The planner is one of two interchangeable engines, dictionary or bit-vector, behind a common interface.
- **PlanExecutor** - The executor steps through the action list, routing each grounded action to a developer-supplied handler matched by name prefix. Handler failure halts the plan, and the caller decides whether to replan.

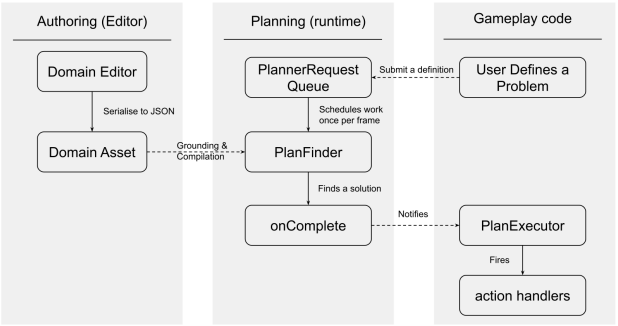

Figure 1: Planning system pipeline.

4 Domain Authoring

4.1 The Domain Asset

A planning domain is stored as a *DomainAsset* ScriptableObject. The asset serializes its content as human-readable JSON through Unity's *ISerializationCallbackReceiver* interface: on save, the in-memory domain model is written to JSON; on load, the JSON is parsed back to the model. The asset also exposes a PDDL export method for use with external tools such as VAL or Fast Downward.

This serialization strategy was chosen deliberately over storing raw PDDL text in the asset. PDDL text is valid for expert users and external tools but brittle as a round-trippable Unity asset format: it has no native support for Unity references, cannot be diffed meaningfully by version control in its raw form when whitespace changes, and requires a PDDL parser on every load. JSON with a fixed schema serializes quickly, produces stable diffs, and keeps the asset readable in a text editor without planning formalism knowledge.

4.2 The Domain Editor

The Domain Editor is a Unity Editor window built with UI Toolkit. It operates on a *DomainAsset* reference dragged into the window or selected via a picker. The window has a left panel showing three sections (types, predicates, actions) and a right panel showing the detail editor for the currently selected item:

- **Type hierarchy** - Types are edited in a *TreeView* that renders the inheritance hierarchy directly. A type's parent type is set via a dropdown in the detail panel. The root type *object* is fixed and cannot be removed. This rendering makes subtype relationships explicit and navigable without understanding PDDL inheritance semantics.
- **Predicate editor** - Each predicate has a name and an ordered parameter list. Parameters have names and types drawn from the type hierarchy. Arity is fixed at definition time; the editor does not permit arity-inconsistent usage sites.
- **Action editor** - Action preconditions are rendered as a recursive visual tree. AND and OR groups are distinct bordered containers. NOT is a first-class node type rendered as an IS/NOT toggle on the predicate row, so negation is flipped in place and its scope stays unambiguous.. Adding a condition opens a *GenericMenu* with four options (Predicate, AND group, OR group) separated into leaf and group categories by a native menu separator; this is not achievable with *DropdownField.choices* in Unity UI Toolkit, which does not support non-selectable separator entries.

There is a support for inline creation **-** any dropdown that selects a type or predicate includes a "Create new" item below a separator. Selecting it creates the entity immediately, selects it in the left panel, and opens its name field for editing, all within the same editor session. This eliminates the context-switch cost of navigating away from an action editor to define a missing type.

To improve user experience, every mutation wraps the domain asset in *Undo.RecordObject*, ensuring that Ctrl-Z

(CMD-Z on Mac OS) works correctly for all operations. This is not optional for Unity Editor tools.

The editor targets STRIPS-style domains with disjunctive and negated preconditions: types with single inheritance, parameterised boolean predicates, and actions whose preconditions are arbitrary AND/OR/NOT trees over predicate literals. PDDL features outside this subset are not supported by the editor: numeric fluents (*:functions*), durative actions, conditional effects (*when*), quantified preconditions (*forall*, *exists*), action costs, and derived predicates (*:derived*). This matches what the runtime planner accepts; importing PDDL files that use unsupported features will fail validation rather than silently dropping features at runtime.

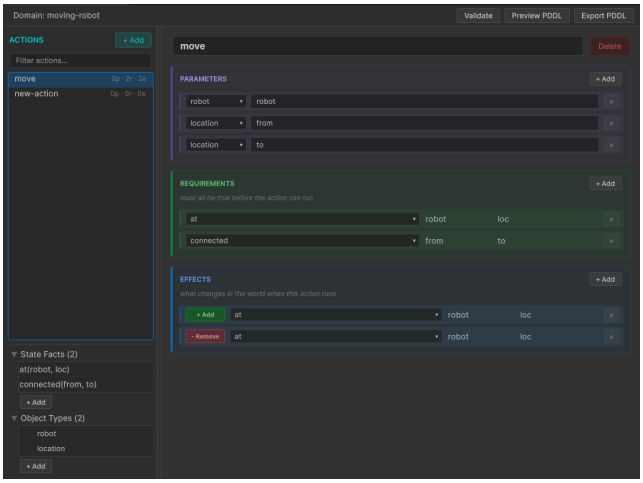

Figure 2: Screenshot of the Domain Editor window showing an action being edited.

4.3 Validation

The editor checks a domain at two levels. Ten structural rules run reactively, after every edit, and report issues inline on the affected action. They catch blank or duplicate names, references to undefined types or predicates, arity mismatches, arguments not bound by the action's parameters, contradictory preconditions (a predicate required both true and false), canceling effects (the same predicate added and removed), duplicate conditions or effects, no-op effects, and actions with no effects. Each rule reports an error or a warning, errors marking a domain that cannot plan meaningfully and warnings flagging likely mistakes. The rules are small and independent, each implementing a single interface, so the rule set is unit-testable and straightforward to extend.

A second, on-demand pass sends the domain, together with a minimal generated problem, through VAL [Howey et al. 2004] for full type-checking, and reports its output. The reactive rules give immediate authoring-time feedback on common mistakes, and the VAL pass is the authoritative external check.

5 Problem Formulation

A planning problem specifies which objects exist and what is currently true, relative to a domain. In game AI, this information lives in the game world (unit positions, inventory states, health values) and must be extracted fresh for each planning request.

Problems are formulated in code through a fluent builder on a *PlanningProblemDefinition* class:

Listing 1: Problem definition builder

```
var problem =
PlanningProblemDefinition.Builder
      ("worker-problem")
.Object("worker-01", "worker")
.Object("log-01", "resource")
.Initially("at", "worker-01",
   "forest-north")
.Initially("resource-at", "log-01",
   "forest-north")
.InitiallyNot("has-resource", "worker-01",
   "log-01")
.Goal("delivered", "log-01")
.Build();
```

Predicate names and object names are plain strings. The builder expands them into grounded predicate keys internally, so the developer writes predicates the same way they appear in the domain definition. For the bit-vector planner, the domain compiler assigns each key a bit position, so runtime predicate access becomes a word-array lookup rather than a hash lookup. The dictionary planner keeps the string-keyed state directly, and is the option when a domain needs typed numeric or string values.

The package also provides a *ProblemAsset* ScriptableObject for problems that are static (tutorial levels, scripted scenarios) and a *PddlProblemImporter* for loading standard PDDL problem files. The code-based formulation is intended for the common game AI case where the problem changes every frame.

Object names are free-form strings chosen by the developer. The planner does not maintain a type registry at runtime; type checking is the domain editor's responsibility at authoring time. At runtime, grounding is the developer's responsibility: *state.SetPredicate("at", true, "worker-01", "forest-north")* is valid regardless of whether *worker-01* was declared as a *worker* type in the domain. This is a deliberate trade-off: runtime type checking would require maintaining type tables that cost memory and complicate the problem API, and the authoring-time validation in the editor catches the class of errors that runtime checking would redundantly catch again.

6 Real-Time Planner Integration

6.1 The Request Queue

Planning requests are submitted to a *PlannerService*, which holds a thread-safe priority queue. Each request carries a *PlanningProblemDefinition*, a numeric priority, an optional owner key, and a callback for the result:

Listing 2: Enqueuing a problem

```
queue.Submit(problem,
    onComplete: result =>
    {
        if (result.Success)
            executor.StartPlan(result);
    },
    priority: 10,
    ownerKey: "worker-01",
    replaceOwner: true);
```

Submit returns immediately and does not block. The request is planned on a later frame, one per frame (Section 6.2), and the *PlanResult* is delivered to the request's callback when the search completes. When *replaceOwner* is set and a request with the same owner key is still pending, the pending one is cancelled and replaced. This models the common pattern where an NPC's situation changes before its previous request has been planned: the stale request is discarded rather than producing a plan for a world state that no longer exists.

Within a priority level, requests are served first in, first out, using a monotonic sequence counter, so a high-priority unit, for example the player's current target, is planned before low-priority background units, without starvation.

6.2 Player Loop Integration

Unity's default *MonoBehaviour.Update* callback runs on the main thread, but the game loop has several phases before and after *Update* that are less commonly used. A custom planning tick is injected into the *PreUpdate* phase using Unity's *PlayerLoop* API:

Listing 3: PlayerLoop integration

```
var planningSystem =
new PlayerLoopSystem
{
    type = typeof(PlanningPlayerLoop),
};
AppendToPhase(ref loop,
typeof(PreUpdate),
planningSystem);
PlayerLoop.SetPlayerLoop(loop);
```

The *PreUpdate* phase runs before *Update* and before physics. Injecting here means planning results are available before any *MonoBehaviour.Update* code runs in a given frame, avoiding one frame of lag between a plan becoming ready and an NPC beginning to act on it.

The *PlanningPlayerLoop.Tick* event fires each *PreUpdate*. A *PlannerService* listens on this event, dequeues one request per frame from the *PlannerRequestQueue*, runs the planner synchronously, and dispatches the result via callback. All execution stays on the main thread: *Submit* is non-blocking, but "asynchronous" here means **same thread, later frame** rather than **another thread**.

Because the callback fires inside *PreUpdate* (before *Update* and physics), an NPC whose plan completes in frame **N+1** can act on it within the same frame; there is no one-frame lag between a plan becoming ready and its first execution step.

This single-threaded design was chosen for the initial implementation. Unity's *JobSystem* and *Burst* compiler are the natural route to parallel planning and a future version could dispatch multiple plans per frame across worker threads, completing them before the next *PreUpdate*.

The one-request-per-frame constraint of the current design means that if multiple NPCs request plans in the same frame, they are served in priority order across successive frames.

6.3 Planner's Internal Representation

The two forward planners run on a single generic A* engine, specialized at compile time to each planner's state representation. They share the search loop and the open- and closed-set machinery and run the same heuristic algorithms; only the state representation differs, so the timing gap between them (Section 8) isolates that representation's per-node cost, which makes the dictionary planner a clean baseline for what compilation contributes.

The dictionary planner keeps a string-keyed map of predicate values. It needs no preprocessing, is usable immediately after grounding, and can hold typed numeric and string values that the bit-vector representation cannot, but on every node it pays for hash lookups, per-successor map cloning, and poor data locality. The bit-vector planner instead compiles the grounded domain into a fixed-width ulong[], with four precomputed bitmasks per action (positive and negative preconditions, add and delete effects) and an achiever index for the heuristic; applicability and transition then become branch-free bitwise operations over a few contiguous words, with no per-node allocation. Grounding and compilation depend only on the domain and the problem's objects, so they run once per object set, are cached, and are reused across replans, and each replan pays only for search; the dictionary planner has no compile step, so its setup is grounding alone, and Section 8 reports the two separately.

6.4 Heuristic Selection

Two heuristics span the trade-off between computation cost and search guidance. The goal-count heuristic (**UnmetGoalCount**) sets h to the number of unsatisfied

goal predicates: it needs no precomputed index, costs almost nothing to evaluate, and is admissible when no single action satisfies more than one goal predicate at once, but its guidance is coarse. **HAddLite** is a more informed, lightweight additive estimate: for each unsatisfied goal predicate it queries the achiever index, picks the action that satisfies it with the fewest currently unmet preconditions, and adds one plus that count to the sum. It is a non-recursive, depth-1 truncation of the additive heuristic h_add [Bonet and Geffner 2001]: where h_add charges each precondition the recursively computed cost of achieving it, to a fixpoint, HAddLite charges each unmet precondition a flat one and stops. It builds no relaxed planning graph, so it is also cheaper, and weaker, than the relaxed-plan heuristic h_FF. Like h_add it is inadmissible, since it treats goal predicates as independent and may double-count shared preconditions, so the planner pairs it with the dominance duplicate-handling mode; inadmissibility is often acceptable for game AI, where any goal-reaching plan is usable.

On the small game domains the two heuristics perform almost identically. The gap appears with harder search: on gripper, goal-count's weak guidance slows the dictionary planner by more than an order of magnitude (about 587 ms versus 24 ms at four balls), while the bit-vector planner, with its far cheaper per-node cost, is affected much less (about 1.2 ms versus 0.6 ms).

7 Plan Execution

7.1 The Executor Model

A *PlanExecutor* manages the lifecycle of a plan. Developers register handler objects against action name prefixes:

Listing 4: Registering executors

```
executor.RegisterExecutor("move",
    new MoveActionExecutor(navAgent));
executor.RegisterExecutor("gather",
    new GatherActionExecutor(animator));
executor.RegisterExecutor("deliver",
    new DeliverActionExecutor(stockpile));
```

When a plan arrives, *StartPlan(result)* begins execution at the first action. On each call to *executor.Update()* (typically from *MonoBehaviour.Update*), the executor asks the current handler whether it has completed or failed. On completion, it advances to the next action. On failure, it raises *OnPlanFailed* and halts. Execution is separate from finding: each agent steps its own *PlanExecutor* every frame, so many plans execute concurrently, while finding is serialized to one per frame (Section 6.2).

Handler lookup is by prefix: *action.Name.StartsWith(registeredPrefix)*. This allows one handler to cover multiple grounded instances of the same action schema (*move(worker-01, base, forest)* and *move(worker-01, forest, mine)* both route to the *move* executor) without requiring the developer to enumerate all groundings at registration time.

The *IActionExecutor* interface is deliberately minimal:

Listing 5: IActionExecutor interface

```
public interface IActionExecutor
{
    bool CanExecute
                (GroundedAction action);
    void StartExecution
                (GroundedAction action);
    bool IsComplete();
    bool HasFailed();
    void Cancel();
}
```

CanExecute is checked before *StartExecution*. If it returns false (for example, because a navigation target has become invalid since the plan was generated), the executor raises *OnActionFailed* immediately rather than attempting execution. This is the primary recovery hook: the developer's handler code can detect that the world has drifted from the plan's assumptions and signal failure, triggering replanning at the caller's discretion.

7.2 World Model Synchronization

The planner's world model is the *PlanningState* snapshot submitted with the problem, and it does not update as the plan executes. This is correct for actions that succeed: the plan assumes each action's effects would be applied, and if execution confirms them, the world matches. It breaks when external events change the world between planning and execution.

The system does not solve this problem automatically but it exposes it. When *OnActionFailed* fires, the handler receives the GroundedAction that failed and a reason, and the expected response is to re-extract world state and submit a new request. A validator is also available that replays the remaining plan against a supplied current state and reports the first action whose preconditions no longer hold, or that the goal is no longer reached; it is the same check used to verify plans in Section 8. The executor does not run it automatically, however, so proactive in-flight drift detection is opt-in and remains future work (Section 9). This one-shot model, plan, execute, and replan if needed, is simpler than maintaining a continuously updated world model, and it suffices when planning frequency can match the rate of world change.

For NPCs that need to respond to world changes mid-plan (such as a guard that receives new information while navigating), the caller can cancel the current plan explicitly via *executor.Cancel()* and submit a new request with the updated state.

8 Evaluation

8.1 Test Domains

The planner is evaluated on three purpose-built game domains and one standard scalable benchmark. The three game domains are archetypal game-AI scenarios, designed to reflect the predicate counts, branching factors, and plan lengths typical of game AI rather than the logistics or scheduling shapes common in planning research, and chosen to span a range of plan lengths and orderings:

- **Soldier** (3-step plan): A soldier advances through two connected locations and eliminates an enemy using direct fire. The plan is fully sequential with no independent steps.
- **Crafter** (7-step plan): A crafter gathers raw materials at a wilderness location and processes them through a crafting chain at a workshop to produce an iron sword. Two preparation actions are mutually independent; the remaining five steps are strictly ordered.
- **Heist** (11-step plan): A thief scouts a location, acquires equipment, disables security, infiltrates a vault, cracks a safe, steals a jewel, distracts a guard, and escapes. Three preparation actions are mutually independent and one action is loosely constrained to precede exit only, while the remaining steps form a strict sequential chain.

These three are deliberately small and lightly branching, which is representative of game AI but limited as a stress test. To probe how the planners behave as a problem scales, the evaluation adds gripper, a standard benchmark from the first International Planning Competition: a robot with two grippers must carry a number of balls between two rooms. Gripper has higher branching and a tunable size, and its symmetry is known to challenge different heuristics, so it exposes differences between the planners and the heuristics that the game domains do not reach.

All domain and problem PDDL files are in **Appendix A**, and example plans, including the Crafter solution, are in **Appendix B**.

8.2 Timing Results

All measurements use Unity Performance Testing (Unity 6000.1.17f1) on Intel Core i7-6700K, with 50 measurements after 10 warm-up runs; reported values are medians in milliseconds. Both forward planners are evaluated with both heuristics, and ReGoap uses its built-in goal-count heuristic. Search and setup are reported separately: search is measured over an already-grounded and, for the bit-vector planner, already-compiled domain, while setup is what each planner does once per object set, grounding for the dictionary planner and grounding plus bitmask compilation for the bit-vector planner. All planners find valid plans of equal length on every domain, and orderings differ only where actions are logically independent. Plan validity was checked independently by replaying each plan over the dictionary state representation (Section 7.2).

Domain	BitVector		Dictionary		ReGoap	Steps
	HAddLite	GoalCount	HAddLite	GoalCount		
Soldier	0.20	0.17	0.23	0.13	0.30	3
Crafter	0.15	0.13	0.60	0.51	65.59	7
Heist	0.15	0.15	1.51	1.37	1591.59	11
Gripper	0.59	1.24	23.54	587.62	N/A	11

Table 1a. Search time (median ms) and plan length (steps)

The bit-vector planner with HAddLite plans in well under a millisecond on every domain. The dictionary planner needs no compilation but pays a higher per-node cost, and the gap widens with difficulty: about 10x on Heist (0.15 ms versus 1.51 ms) and about 40x on gripper (0.59 ms versus 23.54 ms). ReGoap degrades sharply on the larger domains, 66 ms on Crafter and about 1.6 s on Heist, and it does not complete gripper at all within budget.

On the three game domains the two heuristics expand essentially the same number of nodes, so they act as near-perfect estimators there, and the small timing differences reflect per-node cost rather than search effort. On gripper this breaks down: goal-count gives little gradient, so the dictionary planner with goal-count is more than an order of magnitude slower than with HAddLite (587.62 ms versus 23.54 ms), while the bit-vector planner, with its cheaper per-node cost, is affected far less (1.24 ms versus 0.59 ms).

Domain	Grounding	Grounding + bitmask compile
Soldier	0.46	0.75
Crafter	0.25	0.36
Heist	0.16	0.23
Gripper	0.81	1.13

Table 1b. Setup time (median ms), once per object set

Setup is relatively fast and dominated by grounding. The bitmask compilation the bit-vector planner adds on top of grounding is small (for example Heist, 0.16 ms grounding versus 0.23 ms grounding plus compilation), and it is paid once per object set and reused across replans, so a frequently replanning agent pays only for search. Full per-test statistics are in Appendix C.

8.3 Request Queue Behavior

The PlannerService decouples plan submission from plan delivery. When several agents submit requests in the same frame, the requests are ordered by priority and dispatched one per frame, which spreads the cost of many concurrent plans across successive frames rather than concentrating it in a single spike. This suits game AI, where a plan rarely needs to be ready in the frame it is requested: a short delay

is acceptable as long as an agent keeps executing its current behavior while it waits. It mirrors Unity's built-in pathfinding, where path requests are processed asynchronously and agents respond after a slight delay. For example, when ten agents submit in the same frame, all ten plans are delivered within ten frames, one per frame, with the highest-priority request served first.

9 Discussion And Future Work

9.1 What This Enables

The primary value of the system is that a developer can go from zero to a working planning-based NPC in the time it takes to author the domain rather than the time it takes to build infrastructure. For experienced developers, that turns planning from a multi-week project into a multi-day one; for less experienced developers, it makes planning accessible at all.

A secondary benefit appeared in early practice: the domain editor makes an agent's reasoning model inspectable and modifiable by people who did not write the code. A designer can open the domain asset, read the action preconditions, and understand why an NPC does what it does, which is essentially impossible with a behavior tree authored by a programmer.

9.2 What This Does Not Solve

The bit-vector representation is boolean. Domains that need health thresholds, resource quantities, or distances must discretize them into predicates such as *health-low* and *health-critical*, or defer them to the executor's *CanExecute* check. This handles coarse decisions but not fine-grained numeric reasoning; the dictionary planner can store typed numeric values, but the planner does not yet search or optimize over them (Section 9.4).

Game worlds are partially observable, but the planner assumes the initial state fully describes the world. Deciding what to include in the problem state is the developer's responsibility, and the system offers no mechanism for reasoning under uncertainty.

The system plans for one agent at a time. Multi-agent planning needs either centralized planning, which is exponential in agent count, or independent planning with shared-state conflict detection; the common game case is handled, where each NPC plans independently and agents interact through world state rather than joint plans.

The executor does not monitor an executing plan for world-state drift. When a step fails, *OnPlanFailed* fires and the developer decides whether to submit a new request. A validator that re-checks the remaining plan against the current state is provided and is used to verify plans (Section 7.2), but wiring it into the executor for proactive replanning is opt-in and not done by default.

No tooling is provided for inspecting why a planning attempt failed, or how much of the search budget it consumed.

9.3 Relationship to PDDL

The domain asset both imports and exports PDDL, so the editor can serve as a standalone PDDL authoring tool, domains can be brought in from or sent to external planners, and domains can be validated with external tools such as VAL. The runtime planner does not consume PDDL directly; it works from the compiled form derived from the JSON asset, grounded actions and, for the bit-vector planner, the bitmask domain. The PDDL subset the editor and runtime jointly cover is given in Section 4.2: features outside that subset cannot be authored, compiled, or executed by this system.

9.4 Future Directions

Several near-term extensions follow from the limitations above. The single-search-per-frame design (Section 6.2) could be lifted for harder plans by time-slicing the search into an interruptible, anytime planner whose work resumes across frames, or by running the search on a worker thread, for example with Unity's job system, completing it before the next tick. Either keeps a heavy search from blocking the main thread.

Numeric reasoning is another clear gap. Adding numeric fluents and an objective metric would let designers express soft goals such as reaching an objective while minimizing the opponent's remaining health, spending the fewest resources, or minimizing time exposed, which the current boolean model can only approximate through preconditions. The dictionary planner's typed state is a natural starting point.

Proactive replanning is within reach: the validator that re-checks the remaining plan (Section 7.2) could be wired into the executor as an opt-in mode that revalidates against the live world and triggers a new request on drift, rather than waiting for a step to fail.

Hierarchical task network (HTN) planning offers more direct designer control over plan structure than STRIPS-style forward search, with natural support for hierarchical task decomposition; HTN planners have shipped in titles from Guerrilla Games (e.g. Killzone 2, and Horizon Forbidden West) and Bitpart AI recently demonstrated a combination of HTNs with generative AI application for NPC social behavior (Orkin 2025).. The editor could gain method authoring and HDDL support, and the executor is already plan-agnostic, so it would need minimal change to run HTN plans.

Finally, scenarios involving fog of war, deception, or stealth require reasoning about what agents know and what they know about others' knowledge. Epistemic planning has been studied academically, including in game-like settings (Eger 2017), but not applied to real-time game

NPC AI; the authoring model could gain epistemic predicates and the forward search could operate over belief states, though heavy domain restriction would be needed to stay within the frame budget.

10 Conclusions

This paper presents a Unity system that addresses the four infrastructure problems that keep planning from being routinely used in game AI: domain authoring, problem formulation, real-time planner integration, and plan execution. The components form a cohesive, wired pipeline, with each stage's output the next stage's input and no glue code required for the common case. Planning is provided by two forward planners over a shared A* engine: a compiled bit-vector planner that reaches sub-millisecond median search times on representative game domains, well within a main-thread frame budget, and a dictionary planner that serves as a baseline isolating what compilation contributes. Gripper served as a scalable benchmark that shows where the planners and the heuristics begin to diverge, and where ReGoap ceases to scale. A priority queue spreads the cost of concurrent planning across frames, making multi-agent planning practical without dedicated threading, and the domain editor makes an agent's reasoning model directly inspectable, lowering the expertise threshold for authoring and understanding planning behaviour in production.

The central argument is that planning adoption in games is a toolchain problem rather than an algorithmic one. Planning algorithms have been capable enough for at least a decade; what has been missing is the end-to-end infrastructure that makes those algorithms the path of least resistance for game developers. Practical contributions of this kind, grounded in real engineering constraints, may be a productive point of interaction between the game AI and automated planning communities.

References

Bonet, B., and Geffner, H. 2001. Planning as Heuristic Search. Artificial Intelligence 129(1–2): 5–33.

De Pellegrin, E. 2020. PDSim: Planning domain simulation with the Unity game engine. In Proceedings of the ICAPS 2020 Workshop on Knowledge Engineering for Planning and Scheduling (KEPS).

Eger, M. 2017. Intentional agents for epistemic games. In Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, vol. 13, 280–282.

Game Developers Conference. 2026. State of the Game Industry 2026. Annual Survey, January 29, 2026. San Francisco: Informa.

Guerrilla Games. 2017. HTN planning in Decima. Technical blog post. Guerrilla Games.

Howey, R.; Long, D.; and Fox, M. 2004. VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL. In Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI 2004), 294–301.

Koenig, S., and Likhachev, M. 2002. D* Lite. In Proceedings of the 18th National Conference on Artificial Intelligence (AAAI 2002), 476–483.

Likhachev, M.; Gordon, G.; and Thrun, S. 2004. ARA*: Anytime A* with provable bounds on sub-optimality. In *Advances in Neural Information Processing Systems*, vol. 16.

Neufeld, X. 2017. Building a planner: A survey of planning systems used in commercial video games. IEEE Transactions on Computational Intelligence and AI in Games.

Orkin, J. 2006. Three states and a plan: The A.I. of F.E.A.R. Game Developers Conference Proceedings.

Orkin, J. 2025. 1 trillion parameters and no plans. Presentation at the AI and Games Conference, London.

Park, J. S.; O'Brien, J. C.; Cai, C. J.; Morris, M. R.; Liang, P.; and Bernstein, M. S. 2023. Generative agents: Interactive simulacra of human behavior. In Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST 2023).

ReGoap. 2016. ReGoap: Goal Oriented Action Planning library for Unity. GitHub repository. https://github.com/luxkun/ReGoap.

Unity Technologies. 2021. AI Planner package, version 0.3.0-preview.3. Experimental release, discontinued. Unity Technologies.

Valmeekam, K.; Olmo, A.; Sreedharan, S.; and Kambhampati, S. 2023. On the planning abilities of large language models: A critical investigation. In Advances in Neural Information Processing Systems, vol. 36.

van der Sterren, W. 2014. Squad tactics: Planned and opportunistic. In Game AI Pro, chap. 36. CRC Press.

A Appendix: Test Game Domains and Problems in PDDL

PDDL representations of three game-AI planning scenarios (Soldier, Crafter, and Heist) used in benchmarks and unit tests, as well as the IPC staple - the Gripper.

A.1 Soldier

A tactical combat scenario. A soldier must advance through connected locations and eliminate an enemy using direct fire. The domain also models cover and reloading for richer variants.

```
(define (domain soldier)
  (:requirements :strips :typing :negative-preconditions)
  (:types agent enemy weapon location)
  (:predicates
    (at ?a - agent ?l - location)
    (connected ?from - location ?to - location)
    (in-cover ?a - agent)
    (cover-at ?l - location)
    (enemy-at ?e - enemy ?l - location)
    (enemy-alive ?e - enemy)
    (has-weapon ?a - agent ?w - weapon)
    (weapon-loaded ?w - weapon)
    (has-grenade ?a - agent)
  )
  (:action move
    :parameters (?a - agent ?from - location ?to - location)
    :precondition (and (at ?a ?from) (connected ?from ?to))
    :effect (and (not (at ?a ?from)) (at ?a ?to) (not (in-cover ?a)))
  )
  (:action take-cover
    :parameters (?a - agent ?l - location)
    :precondition (and (at ?a ?l) (cover-at ?l) (not (in-cover ?a)))
    :effect (in-cover ?a)
  )
  (:action reload
    :parameters (?a - agent ?w - weapon)
    :precondition (and (has-weapon ?a ?w) (in-cover ?a) (not (weapon-loaded ?w)))
    :effect (weapon-loaded ?w)
  )
  (:action shoot
    :parameters (?a - agent ?w - weapon ?e - enemy ?l - location)
    :precondition (and (at ?a ?l) (has-weapon ?a ?w) (weapon-loaded ?w) (enemy-at ?e
?l) (enemy-alive ?e))
    :effect (and (not (enemy-alive ?e)) (not (weapon-loaded ?w)))
  )
  (:action throw-grenade
    :parameters (?a - agent ?from - location ?to - location ?e - enemy)
    :precondition (and (at ?a ?from) (has-grenade ?a) (connected ?from ?to)
(enemy-at ?e ?to) (enemy-alive ?e))
    :effect (and (not (enemy-alive ?e)) (not (has-grenade ?a)))
  )
)
```

```
(define (problem soldier-problem)
  (:domain soldier)
  (:objects
    soldier1 - agent
    enemy1 - enemy
    rifle1 - weapon
    entrance corridor bunker - location
  )
  (:init
    (at soldier1 entrance)
    (has-weapon soldier1 rifle1)
    (weapon-loaded rifle1)
    (enemy-at enemy1 bunker)
    (enemy-alive enemy1)
    (connected entrance corridor)
    (connected corridor bunker)
  )
  (:goal
    (not (enemy-alive enemy1))
  )
)
```

A.2 Crafter

A crafting-chain scenario. An agent must gather raw materials and process them through a sequence of dependent crafting steps to produce a final item.

```
(define (domain crafter)
  (:requirements :strips :typing :negative-preconditions)
  (:types agent location)
  (:predicates
    (at ?a - agent ?l - location)
    (connected ?from - location ?to - location)
    (has-wood ?a - agent)
    (has-planks ?a - agent)
    (has-iron-ore ?a - agent)
    (has-iron-ingot ?a - agent)
    (has-iron-sword ?a - agent)
    (has-pickaxe ?a - agent)
    (workbench-at ?l - location)
    (furnace-at ?l - location)
    (wood-available ?l - location)
    (ore-available ?l - location)
  )
  (:action travel
    :parameters (?a - agent ?from - location ?to - location)
    :precondition (and (at ?a ?from) (connected ?from ?to))
    :effect (and (not (at ?a ?from)) (at ?a ?to))
  )
  (:action gather-wood
    :parameters (?a - agent ?l - location)
    :precondition (and (at ?a ?l) (wood-available ?l) (not (has-wood ?a)))
    :effect (and (has-wood ?a) (not (wood-available ?l)))
  )
```

```
  (:action mine-ore
    :parameters (?a - agent ?l - location)
    :precondition (and (at ?a ?l) (ore-available ?l) (has-pickaxe ?a) (not
(has-iron-ore ?a)))
    :effect (and (has-iron-ore ?a) (not (ore-available ?l)))
  )
  (:action craft-planks
    :parameters (?a - agent ?l - location)
    :precondition (and (at ?a ?l) (has-wood ?a) (workbench-at ?l))
    :effect (and (has-planks ?a) (not (has-wood ?a)))
  )
  (:action build-furnace
    :parameters (?a - agent ?l - location)
    :precondition (and (at ?a ?l) (has-planks ?a) (workbench-at ?l) (not (furnace-at
?l)))
    :effect (and (furnace-at ?l) (not (has-planks ?a)))
  )
  (:action smelt-ingot
    :parameters (?a - agent ?l - location)
    :precondition (and (at ?a ?l) (has-iron-ore ?a) (furnace-at ?l))
    :effect (and (has-iron-ingot ?a) (not (has-iron-ore ?a)))
  )
  (:action forge-sword
    :parameters (?a - agent ?l - location)
    :precondition (and (at ?a ?l) (has-iron-ingot ?a) (workbench-at ?l) (furnace-at
?l))
    :effect (has-iron-sword ?a)
  )
)
```

```
(define (problem crafter-problem)
  (:domain crafter)
  (:objects
    crafter1 - agent
    wilderness workshop - location
  )
  (:init
    (at crafter1 wilderness)
    (has-pickaxe crafter1)
    (wood-available wilderness)
    (ore-available wilderness)
    (connected wilderness workshop)
    (workbench-at workshop)
  )
  (:goal
    (has-iron-sword crafter1)
  )
)
```

A.3 Heist

A stealth scenario. A thief must scout the location, acquire equipment, disable security, infiltrate a vault, crack the safe, steal a jewel, distract a guard, and escape. The domain has a mix of independent preparation actions and a strictly ordered endgame.

```
(define (domain heist)
  (:requirements :strips :typing :negative-preconditions)
  (:types agent)
  (:predicates
    (location-cased ?a - agent)
    (has-lockpicks ?a - agent)
    (disguised ?a - agent)
    (camera-disabled ?a - agent)
    (vault-unlocked ?a - agent)
    (in-vault ?a - agent)
    (safe-open ?a - agent)
    (has-jewel ?a - agent)
    (has-noise-maker ?a - agent)
    (guard-distracted ?a - agent)
    (escaped ?a - agent)
  )
  (:action case-location
    :parameters (?a - agent)
    :precondition (not (location-cased ?a))
    :effect (location-cased ?a)
  )
  (:action acquire-lockpicks
    :parameters (?a - agent)
    :precondition (not (has-lockpicks ?a))
    :effect (has-lockpicks ?a)
  )
  (:action don-disguise
    :parameters (?a - agent)
    :precondition (not (disguised ?a))
    :effect (disguised ?a)
  )
  (:action disable-camera
    :parameters (?a - agent)
    :precondition (and (location-cased ?a) (not (camera-disabled ?a)))
    :effect (camera-disabled ?a)
  )
  (:action pick-lock
    :parameters (?a - agent)
    :precondition (and (has-lockpicks ?a) (camera-disabled ?a) (not (vault-unlocked
?a)))
    :effect (vault-unlocked ?a)
  )
  (:action enter-vault
    :parameters (?a - agent)
    :precondition (and (disguised ?a) (vault-unlocked ?a) (not (in-vault ?a)))
    :effect (in-vault ?a)
  )
  (:action crack-safe
    :parameters (?a - agent)
```

```
    :precondition (and (in-vault ?a) (not (safe-open ?a)))
    :effect (safe-open ?a)
  )
  (:action steal-jewel
    :parameters (?a - agent)
    :precondition (and (in-vault ?a) (safe-open ?a) (not (has-jewel ?a)))
    :effect (has-jewel ?a)
  )
  (:action use-noise-maker
    :parameters (?a - agent)
    :precondition (and (has-noise-maker ?a) (not (guard-distracted ?a)))
    :effect (and (guard-distracted ?a) (not (has-noise-maker ?a)))
  )
  (:action exit-vault
    :parameters (?a - agent)
    :precondition (and (in-vault ?a) (guard-distracted ?a) (has-jewel ?a))
    :effect (not (in-vault ?a))
  )
  (:action escape
    :parameters (?a - agent)
    :precondition (and (not (in-vault ?a)) (has-jewel ?a))
    :effect (escaped ?a)
  )
)
```

```
(define (problem heist-problem)
  (:domain heist)
  (:objects
    thief1 - agent
  )
  (:init
    (has-noise-maker thief1)
  )
  (:goal
    (escaped thief1)
  )
)
```

A.4 Gripper

A standard planning-competition benchmark, included to test how the planners scale, rather than a purpose-built game scenario. A robot with two grippers must carry a number of balls from one room to another. The two interchangeable grippers and interchangeable balls create symmetry that drives an unnecessary combinatorial blow-up in many planners, and the problem size is tunable by the number of balls, which makes it a useful stress test that the small game domains do not reach.

IPC 1998, Gripper, Round 1, STRIPS track.
Author: Jana Koehler. Original name `prob01.pddl`

```
(define (domain gripper-strips)
   (:predicates (room ?r)
        (ball ?b)
```

```
        (gripper ?g)
        (at-robby ?r)
        (at ?b ?r)
        (free ?g)
        (carry ?o ?g))

   (:action move
       :parameters  (?from ?to)
       :precondition (and  (room ?from) (room ?to) (at-robby ?from))
       :effect (and  (at-robby ?to)
             (not (at-robby ?from))))

   (:action pick
       :parameters (?obj ?room ?gripper)
       :precondition  (and  (ball ?obj) (room ?room) (gripper ?gripper)
                (at ?obj ?room) (at-robby ?room) (free ?gripper))
       :effect (and (carry ?obj ?gripper)
            (not (at ?obj ?room))
            (not (free ?gripper))))

   (:action drop
       :parameters  (?obj  ?room ?gripper)
       :precondition  (and  (ball ?obj) (room ?room) (gripper ?gripper)
                (carry ?obj ?gripper) (at-robby ?room))
       :effect (and (at ?obj ?room)
            (free ?gripper)
            (not (carry ?obj ?gripper)))))
```

```
(define (problem strips-gripper-x-1)
   (:domain gripper-strips)
   (:objects rooma roomb ball4 ball3 ball2 ball1 left right)
   (:init (room rooma)
          (room roomb)
          (ball ball4)
          (ball ball3)
          (ball ball2)
          (ball ball1)
          (at-robby rooma)
          (free left)
          (free right)
          (at ball4 rooma)
          (at ball3 rooma)
          (at ball2 rooma)
          (at ball1 rooma)
          (gripper left)
          (gripper right))
   (:goal (and (at ball4 roomb)
               (at ball3 roomb)
               (at ball2 roomb)
               (at ball1 roomb))))
```

B Appendix: Plans Found By Planners

Plans found for domains and problems from Appendix A using different planners and heuristics.

B.1 Solutions to the soldier-problem

	BitVector / HAddLite	BitVector / Unmet	Dictionary / HAddLite	Dictionary / Unmet	ReGoap
1	move(soldier1, entrance, corridor)	move(soldier1, entrance, corridor)	move(soldier1, entrance, corridor)	move(soldier1, entrance, corridor)	move(soldier1, entrance, corridor)
2	move(soldier1, corridor, bunker)	move(soldier1, corridor, bunker)	move(soldier1, corridor, bunker)	move(soldier1, corridor, bunker)	move(soldier1, corridor, bunker)
3	shoot(soldier1, rifle1, enemy1, bunker)	shoot(soldier1, rifle1, enemy1, bunker)	shoot(soldier1, rifle1, enemy1, bunker)	shoot(soldier1, rifle1, enemy1, bunker)	shoot(soldier1, rifle1, enemy1, bunker)

B.2 Solutions to the crafter-problem

	BitVector / HAddLite	BitVector / Unmet	Dictionary / HAddLite	Dictionary / Unmet	ReGoap
1	mine-ore(crafter1, wilderness)	mine-ore(crafter1, wilderness)	gather-wood(crafter1, wilderness)	mine-ore(crafter1, wilderness)	mine-ore(crafter1, wilderness)
2	gather-wood(crafter1, wilderness)	gather-wood(crafter1, wilderness)	mine-ore(crafter1, wilderness)	gather-wood(crafter1, wilderness)	gather-wood(crafter1, wilderness)
3	travel(crafter1, wilderness, workshop)	travel(crafter1, wilderness, workshop)	travel(crafter1, wilderness, workshop)	travel(crafter1, wilderness, workshop)	travel(crafter1, wilderness, workshop)
4	craft-planks(crafter1, workshop)	craft-planks(crafter1, workshop)	craft-planks(crafter1, workshop)	craft-planks(crafter1, workshop)	craft-planks(crafter1, workshop)
5	build-furnace(crafter1, workshop)	build-furnace(crafter1, workshop)	build-furnace(crafter1, workshop)	build-furnace(crafter1, workshop)	build-furnace(crafter1, workshop)
6	smelt-ingot(crafter1, workshop)	smelt-ingot(crafter1, workshop)	smelt-ingot(crafter1, workshop)	smelt-ingot(crafter1, workshop)	smelt-ingot(crafter1, workshop)
7	forge-sword(crafter1, workshop)	forge-sword(crafter1, workshop)	forge-sword(crafter1, workshop)	forge-sword(crafter1, workshop)	forge-sword(crafter1, workshop)

B.3 Solutions to the heist-problem

	BitVector / HAddLite	BitVector / Unmet	Dictionary / HAddLite	Dictionary / Unmet	ReGoap
1	acquire-lockpicks(thief1)	acquire-lockpicks(thief1)	case-location(thief1)	case-location(thief1)	don-disguise(thief1)
2	don-disguise(thief1)	don-disguise(thief1)	disable-camera(thief1)	disable-camera(thief1)	acquire-lockpicks(thief1)
3	case-location(thief1)	case-location(thief1)	acquire-lockpicks(thief1)	acquire-lockpicks(thief1)	use-noise-maker(thief1)
4	disable-camera(thief1)	disable-camera(thief1)	pick-lock(thief1)	pick-lock(thief1)	case-location(thief1)
5	pick-lock(thief1)	pick-lock(thief1)	don-disguise(thief1)	don-disguise(thief1)	disable-camera(thief1)
6	enter-vault(thief1)	enter-vault(thief1)	enter-vault(thief1)	use-noise-maker(thief1)	pick-lock(thief1)

7	crack-safe(thief1)	use-noise-maker(thief1)	crack-safe(thief1)	enter-vault(thief1)	enter-vault(thief1)
8	steal-jewel(thief1)	crack-safe(thief1)	steal-jewel(thief1)	crack-safe(thief1)	crack-safe(thief1)
9	use-noise-maker(thief1)	steal-jewel(thief1)	use-noise-maker(thief1)	steal-jewel(thief1)	steal-jewel(thief1)
10	exit-vault(thief1)	exit-vault(thief1)	exit-vault(thief1)	exit-vault(thief1)	exit-vault(thief1)
11	escape(thief1)	escape(thief1)	escape(thief1)	escape(thief1)	escape(thief1)

C Appendix: Performance Benchmark Statistics

C.1 Planning time performance results

Domain	Planner	Heuristic	Min (ms)	Median (ms)	Max (ms)	Avg (ms)	Std Dev (ms)
Soldier	BitVector	GoalCount	0.15	0.17	0.31	0.18	0.03
		HAddLite	0.16	0.20	0.42	0.21	0.05
	Dictionary	GoalCount	0.11	0.13	0.24	0.13	0.02
		HAddLite	0.19	0.23	0.25	0.23	0.01
	ReGoap	ReGoap	0.27	0.30	0.48	0.34	0.07
Crafter	BitVector	GoalCount	0.13	0.13	0.22	0.14	0.02
		HAddLite	0.13	0.15	0.17	0.15	0.01
	Dictionary	GoalCount	0.41	0.51	0.90	0.53	0.10
		HAddLite	0.49	0.60	0.77	0.59	0.05
	ReGoap	ReGoap	52.16	65.59	102.99	65.74	8.90
Heist	BitVector	GoalCount	0.15	0.15	0.45	0.17	0.05
		HAddLite	0.13	0.15	0.31	0.17	0.05
	Dictionary	GoalCount	1.13	1.37	1.45	1.37	0.06
		HAddLite	1.24	1.51	1.72	1.44	0.14
	ReGoap	ReGoap	1487.61	1591.59	1923.15	1649.01	127.45
Gripper	BitVector	GoalCount	1.08	1.24	2.18	1.31	0.24
		HAddLite	0.58	0.59	1.22	0.62	0.10
	Dictionary	GoalCount	553.28	587.62	795.11	604.44	49.73
		HAddLite	21.68	23.54	39.19	24.86	3.10

C.2 Grounding and compilation performance results

Domain	Operation	Min (ms)	Median (ms)	Max (ms)	Avg (ms)	Std Dev (ms)
Soldier	Grounding + Comp.	0.56	0.75	2.27	0.88	0.31
	Grounding	0.39	0.46	0.53	0.44	0.03
Crafter	Grounding + Comp.	0.35	0.36	0.66	0.40	0.07
	Grounding	0.25	0.25	0.33	0.27	0.02
Heist	Grounding + Comp.	0.19	0.23	0.69	0.25	0.08
	Grounding	0.15	0.16	0.24	0.16	0.01
Gripper	Grounding + Comp.	0.98	1.13	1.29	1.11	0.08
	Grounding	0.69	0.81	1.48	0.9	0.2